

SQLALCHEMY

(DataBase와 ORM)

DataBase와 ORM : FastAPI에서의 활용

- FastAPI에서 데이터베이스 관리와 ORM(Object-Relational Mapping)의 사용은 웹 어플리케이션의 중요한 부분
- ORM은 데이터베이스의 테이블을 객체로 매핑하여, 개발자가 SQL 쿼리 없이 데이터베이스를 쉽게 조작할 수 있게 함

FastAPI와 MySQL 연동 방법

- 필요 라이브러리 설치

- `pip install sqlalchemy==1.4.39`
- `pip install pymysql==1.1.0`

- 데이터베이스 모델 정의

- SQLAlchemy를 사용하여 데이터베이스 모델을 정의
- 각 모델은 데이터베이스의 테이블에 해당하며, 모델의 인스턴스는 테이블의 레코드를 나타냄

FastAPI와 MySQL 연동 방법

```
from fastapi import FastAPI, Depends
from sqlalchemy.orm import Session
from sqlalchemy import Column, Integer, String, create_engine
from sqlalchemy.ext.declarative import declarative_base
from pydantic import BaseModel

DATABASE_URL = "mysql+pymysql://username:password@localhost/fastapi" # 사용자의 데이터베이스 정보로 변경.
engine = create_engine(DATABASE_URL)

Base = declarative_base()

class User(Base):
    __tablename__ = 'users'
    id = Column(Integer, primary_key=True, index=True)
    username = Column(String(50), unique=True, index=True) # 사용자 이름, 중복 불가능하고 인덱싱.
    email = Column(String(120)) # 이메일 주소, 길이는 120자로 제한.
```

FastAPI와 MySQL 연동 방법

- `create_engine` : SQLAlchemy에서 데이터베이스 엔진을 생성하는 데 사용하는 함수
- `DATABASE_URL` : 데이터베이스 접속 정보를 담은 문자열
 - `mysql+pymysql` : 사용할 데이터베이스 종류(mysql)와 Python DBAPI 구현(pymysql)
 - `username:password` : 데이터베이스 사용자 이름과 비밀번호
 - `host` : 데이터베이스 서버의 주소
 - `fastapi` : 접속할 데이터베이스 이름
- `engine = create_engine(DATABASE_URL)` : 설정된 `DATABASE_URL`을 사용하여 데이터베이스 엔진 객체를 생성함.
이 엔진은 데이터베이스와의 모든 통신을 관리함
- 주의 사항
 - 데이터베이스 사용자 이름, 비밀번호, 호스트 및 데이터베이스 이름은 실제 환경에 맞게 변경해야 함
 - 로컬 환경에서 MySQL 서버를 실행하는 경우 `host` 부분은 일반적으로 `localhost`로 설정됨
 - MySQL 서버가 실행 중이며 해당 사용자로 접속 권한이 있는지 확인해야 함

FastAPI와 MySQL 연동 방법

- 데이터베이스 세션 생성 및 관리

```
class UserCreate(BaseModel):  
    username: str  
    email: str  
  
def get_db():  
    db = Session(bind=engine)  
    try:  
        yield db  
    finally:  
        db.close()  
  
Base.metadata.create_all(bind=engine)
```

- Base.metadata.create_all(bind=engine) 코드 관련

- 테이블이 이미 있을 때 : 코드에 create_all 함수가 있더라도 아무런 문제가 발생하지 않음. 왜냐하면 이 함수는 기존에 테이블이 있다면 그냥 무시하기 때문. 그러나 이미 테이블이 있는 상태에서는 이 코드를 실행할 필요는 없음
- 테이블이 없을 때 : 데이터베이스에 테이블이 없는 초기 상태라면 이 함수를 통해 테이블을 생성할 수 있음

FastAPI와 MySQL 연동 방법

- FastAPI 어플리케이션에 모델 통합

```
app = FastAPI()

@app.get("/")
def read_root():
    return {"message": "Hello, World!"}

@app.post("/users/")
def create_user(user: UserCreate, db: Session = Depends(get_db)):
    new_user = User(username=user.username, email=user.email)
    db.add(new_user)      # 생성된 User 인스턴스를 데이터베이스 세션에 추가.
    db.commit()           # 데이터베이스에 대한 변경사항을 커밋.
    db.refresh(new_user)  # 데이터베이스로부터 새 User 인스턴스의 최신 정보를 가져옴.
    return {"id": new_user.id, "username": new_user.username, "email": new_user.email}
```

- 테스트를 위한 데이터베이스 생성

- CREATE DATABASE fastapi DEFAULT CHARSET utf8mb4 COLLATE utf8mb4_general_ci;

FastAPI와 MySQL 연동 방법

- Column에서 사용할 수 있는 옵션

1. Data Types

- `Integer` : 정수형 데이터. 예: `id = Column(Integer)`
- `String` : 문자열 데이터. `String` 은 문자열 최대 길이를 명시해야 합니다. 예: `username = Column(String(50))`
- `Boolean` : 참/거짓을 나타내는 불린형 데이터. 예: `is_active = Column(Boolean)`
- `Float` : 부동소수점 데이터. 예: `grade = Column(Float)`
- `DateTime` : 날짜와 시간을 나타냅니다. 예: `created_at = Column(DateTime)`

2. Primary Key

- `primary_key=True` : 해당 필드가 기본 키임을 명시합니다. 예: `id = Column(Integer, primary_key=True)`

3. Index

- `index=True` : 이 필드에 인덱스를 생성합니다. 예: `username = Column(String(50), index=True)`

FastAPI와 MySQL 연동 방법

- Column에서 사용할 수 있는 옵션

4. Unique

- `unique=True` : 이 필드의 값은 고유해야 함을 명시합니다. 예: `email = Column(String(120), unique=True)`

5. Nullable

- `nullable=False` : 이 필드는 NULL 값을 허용하지 않습니다. 예: `username = Column(String(50), nullable=False)`

6. Default Value

- `default` : 이 필드의 기본값을 설정합니다. 예: `is_active = Column(Boolean, default=True)`

FastAPI와 MySQL 연동 방법

- Column에서 사용할 수 있는 옵션

7. Auto Increment

- `autoincrement=True` : 이 필드의 값을 자동으로 증가시킵니다. 예: `id = Column(Integer, autoincrement=True)`

8. Foreign Key

- `ForeignKey` : 외래 키를 설정할 때 사용됩니다. 예: `parent_id = Column(Integer, ForeignKey('parents.id'))`

9. Comments

- `comment` : 이 필드에 설명을 추가할 수 있습니다. 예: `id = Column(Integer, comment="Primary Key")`

FastAPI와 SQLAlchemy를 사용한 CRUD

- Session의 중요성

- Session은 SQLAlchemy에서 데이터베이스와의 모든 상호 작용을 관리함
- CRUD 작업은 세션을 통해 실제 데이터베이스에 변화를 주거나 정보를 가져올 수 있음

- SQLAlchemy Session 준비

- FastAPI에서 SQLAlchemy와 함께 작업하려면 Session 객체가 필요함
- Session은 SQLAlchemy의 일부로, 데이터베이스와 연결을 관리함
 - `from sqlalchemy.orm import Session`

- 데이터 입력 : Create 작업(두가지 방안 존재 - 배포 코드 참조)

- 첫 번째 방안과 두 번째 방안의 비교
 - 첫 번째 방안(Depends 사용)
 - 자동 세션 관리 : Depends를 통해 세션 생명주기 자동 관리
 - 재사용성 : get_db 함수 재사용 가능
 - 효율적 자원 관리
 - 두 번째 방안(sessionmaker 직접 사용)
 - 명시적 세션 생성 및 종료
 - 코드 명시성 증가
 - 예외 상황에서 세션 닫기를 놓칠 위험 존재

FastAPI와 SQLAlchemy를 사용한 CRUD

- 다양한 검색 문법: SQLAlchemy 사용법

- User 클래스를 기반으로 한 db.query()를 이용한 다양한 SQLAlchemy 검색 문법을 설명
- 모든 사용자의 모든 정보를 조회

```
users = db.query(User).all()
```

- 특정 컬럼(예: 사용자 이름)만 조회

```
usernames = db.query(User.username).all()
```

- 특정 조건(예: 사용자 이름이 'john')을 만족하는 데이터 필터링

```
user = db.query(User).filter(User.username == 'john').first()
```

- 여러 조건을 연결하여 필터링

```
user = db.query(User).filter(User.username == 'john').filter(User.email == 'john@gmail.com').first()
```

- or 조건

```
from sqlalchemy import or_
```

```
user = db.query(User).filter(or_(User.username == 'john', User.email == 'john@gmail.com')).first()
```

FastAPI와 SQLAlchemy를 사용한 CRUD

- 결과를 특정 컬럼(예: 사용자 이름)으로 오름차순 정렬

```
users = db.query(User).order_by(User.username).all()
```

- 결과를 특정 컬럼(예: 사용자 이름)으로 내림차순 정렬

```
from sqlalchemy import desc
```

```
users = db.query(User).order_by(desc(User.username)).all()
```

- 조회 결과의 개수를 제한(예: 상위 5개)

```
users = db.query(User).limit(5).all()
```

- 조회 시작점을 지정하여 결과를 건너뛴(예: 처음 2개 건너뛰기)

```
users = db.query(User).offset(2).all()
```

- 조회 결과의 총 개수를 반환

```
count = db.query(User).count()
```

FastAPI와 SQLAlchemy를 사용한 CRUD

- 그룹화(Group By)

- 목적: 특정 컬럼을 기준으로 데이터를 그룹화하고, 각 그룹에 대한 집계 연산을 수행
- 사용법: query() 메소드에 모델의 필드를 지정하고, group_by() 메소드로 그룹화할 기준을 설정
- Count 사용: 각 그룹에 속한 요소의 개수를 세어 반환

```
from sqlalchemy import func
```

```
db_users_count = db.query(User.username, func.count(User.id)).group_by(User.username).all()
```

- Max 사용: 각 그룹에서 가장 큰 값을 찾음

```
db_users_max = db.query(User.username, func.max(User.id)).group_by(User.username).all()
```

- Min 사용: 각 그룹에서 가장 작은 값을 찾음

```
db_users_min = db.query(User.username, func.min(User.id)).group_by(User.username).all()
```

- Sum 사용: 각 그룹의 값들을 모두 더함

```
db_users_sum = db.query(User.username, func.sum(User.id)).group_by(User.username).all()
```